Algorithmique Fondamentale : Recherche Séquentielle et Recherche Dichotomique dans les Tableaux

Thème principal : Algorithmes de recherche dans les structures de données linéaires et comparaison de complexité temporelle

Synthèse exécutive : Ce cours traite des deux méthodes fondamentales de recherche d'un élément au sein d'un tableau : la recherche séquentielle (linéaire) et la recherche dichotomique (basée sur le principe 'diviser pour régner'). L'enjeu central réside dans l'impact direct de la structure et de l'organisation des données sur l'efficacité algorithmique. Si la recherche séquentielle s'applique à tout tableau de manière universelle mais en complexité linéaire O(n), la recherche dichotomique exige un tableau préalablement trié pour réduire drastiquement l'espace de recherche à chaque itération, atteignant une complexité logarithmique O(log n). L'étudiant doit savoir choisir l'approche pertinente selon le contexte, maîtriser l'évolution pas à pas des pointeurs d'indices (début, fin, milieu) et évaluer la complexité dans le pire des cas.

1. La Recherche Séquentielle (ou Linéaire)

La recherche séquentielle est la méthode la plus intuitive pour trouver un élément dans un tableau. L'algorithme démarre à l'indice 0 et compare la valeur contenue dans chaque case avec la valeur recherchée (la cible). Il avance pas à pas. Deux issues sont possibles : soit l'élément est trouvé et le parcours s'arrête immédiatement en retournant l'indice correspondant, soit la fin du tableau est atteinte sans succès et l'algorithme retourne une valeur conventionnelle d'absence (ex: -1 ou Faux). Si le tableau contient n éléments, le pire des cas exige d'effectuer n comparaisons. La complexité est donc notée O(n). Cette méthode est privilégiée lorsque les données ne sont pas triées et qu'un tri préalable serait plus coûteux que la recherche elle-même.

- Parcours exhaustif du tableau indice par indice, du début (0) jusqu'à la fin (n-1).
- Absence totale de précondition : fonctionne sans condition préalable sur des tableaux triés ou non triés.
- Complexité temporelle linéaire O(n) au pire des cas (élément situé en n-1 ou absent).
- Arrêt anticipé dès que la valeur recherchée est localisée (succès).

Cas concret enseignant : *Le professeur utilise l'analogie de la recherche d'une clé tombée dans un tas de feuilles mortes : comme le tas n'a aucune organisation ou structure logique, la seule façon absolue de trouver la clé est de soulever chaque feuille une à une, de la première jusqu'à la dernière.*

2. La Recherche Dichotomique (Principe du Diviser pour Régner)

La recherche dichotomique exploite l'ordre d'un tableau trié pour éliminer massivement les valeurs candidates. À chaque étape, on définit une zone de recherche délimitée par deux indices, 'debut' et 'fin'. On calcule le milieu de cette zone : milieu = (debut + fin) // 2 (division entière). On compare ensuite l'élément T[milieu] avec la cible : 1) Si T[milieu] == cible, l'élément est trouvé ; 2) Si la cible est strictement inférieure à T[milieu], la cible ne peut se trouver que dans la moitié gauche, on adapte alors le pointeur 'fin = milieu - 1' ; 3) Si la cible est supérieure, on adapte 'debut = milieu + 1'. Le processus s'arrête lorsque la cible est trouvée ou lorsque debut > fin (zone vide, élément absent). En divisant par 2 la zone à chaque étape, on obtient une complexité en O(log2 n). Par exemple, pour un tableau de 1 000 000 d'éléments triés, 20 comparaisons suffisent au maximum.

- Précondition absolue et indispensable : le tableau doit être strictement trié au préalable.
- Paradigme Diviser pour Régner : réduction de 50 % de l'espace de recherche à chaque itération.
- Gestion rigoureuse de trois pointeurs d'indices : debut, fin, et milieu calculé par (debut + fin) // 2.
- Complexité temporelle logarithmique O(log2 n) dans le pire des cas.

Cas concret enseignant : *Le professeur illustre cela avec le jeu du 'Devine un nombre entre 1 et 100' : proposer 1, 2, 3... est une stratégie séquentielle médiocre. Proposer 50 élimine immédiatement la moitié des nombres si la réponse est 'c'est plus grand'. En réitérant cette stratégie, on trouve le nombre exact en 7 étapes maximum (car 2^7 = 128 > 100).*

3. Arbitrage entre Recherche Séquentielle et Dichotomique

En programmation et en algorithmique, choisir le bon algorithme dépend du contexte opérationnel. Bien que la recherche dichotomique soit immensément plus rapide que la recherche séquentielle pour de grands volumes, elle impose le tri préalable du tableau. Le meilleur algorithme de tri (comme le Tri Fusion ou le Tri Rapide) s'exécute en O(n log n). Ainsi, pour effectuer une unique recherche dans un tableau non trié, il est plus performant d'exécuter une recherche séquentielle O(n) que de trier d'abord le tableau en O(n log n) pour ensuite effectuer une recherche dichotomique en O(log n). En revanche, si le tableau est statique et doit subir des milliers de recherches, l'investissement initial du tri est largement amorti.

- Coût du tri préalable : Trier un tableau non trié nécessite O(n log n).
- Règle d'arbitrage : Si une seule recherche est effectuée sur un tableau non trié, la recherche séquentielle O(n) est plus rapide que Trier + Dichotomie.
- Multiplicité des recherches : Si l'on effectue de nombreuses recherches sur un tableau fixe, trier une seule fois le tableau est extrêmement rentable.

Cas concret enseignant : *Analogie du dictionnaire : Organiser les mots par ordre alphabétique demande un travail considérable à l'imprimeur, mais ce tri permet à des millions de lecteurs de trouver un mot en quelques secondes pendant des années.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours utiliser la recherche dichotomique si elle est beaucoup plus rapide que la recherche séquentielle ?
Réponse du professeur : La recherche dichotomique a une précondition stricte : le tableau doit être trié. Si le tableau n'est pas trié, il faut le trier au préalable, ce qui coûte O(n log n) opérations. Pour une seule recherche ponctuelle sur un tableau non trié, la recherche séquentielle O(n) reste donc plus rapide que Tri + Dichotomie.

Question étudiant : Comment calcule-t-on l'indice du milieu si le sous-tableau contient un nombre pair d'éléments ?
Réponse du professeur : On utilise la division entière (troncature vers le bas, notée // en Python). Par exemple, si debut = 0 et fin = 9 (10 éléments), milieu = (0 + 9) // 2 = 4. L'algorithme prend la position 4 sans aucun problème, ce qui découpe le tableau en deux parties équilibrées.

■ Exercice & Démonstration d'Application

Énoncé : Soit le tableau trié T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61] comportant n = 10 éléments (indices de 0 à 9). Appliquer pas à pas l'algorithme de recherche dichotomique pour localiser la valeur 42.

Résolution pas à pas :
Étape 0 (Initialisation) : debut = 0, fin = 9. Étape 1 : - Calcul du milieu : milieu = (0 + 9) // 2 = 4. - Valeur examinée : T[4] = 19. - Comparaison : 42 > 19. La cible est supérieure, on recherche dans la sous-partie droite. - Mise à jour du pointeur debut : debut = milieu + 1 = 5. Étape 2 : - Zone de recherche : debut = 5, fin = 9. - Calcul du milieu : milieu = (5 + 9) // 2 = 7. - Valeur examinée : T[7] = 42. - Comparaison : T[7] == 42. La valeur recherchée est trouvée !

Résultat : La valeur 42 a été localisée à l'indice 7 en seulement 2 comparaisons, alors qu'une recherche séquentielle aurait nécessité 8 comparaisons (du tableau[0]=3 jusqu'à tableau[7]=42).

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la condition impérative indispensable pour pouvoir appliquer une recherche dichotomique sur un tableau ?

R : Le tableau doit impérativement être trié au préalable selon un ordre total (ex: croissant).

Q : Quelle est la complexité temporelle au pire des cas d'une recherche séquentielle sur un tableau de taille n ?

R : La complexité temporelle au pire des cas est linéaire, notée O(n).

Q : Combien de comparaisons au maximum la recherche dichotomique nécessite-t-elle pour localiser un élément dans un tableau trié de 1024 éléments ?

R : Elle nécessite au maximum 10 comparaisons, car log2(1024) = 10 (puisque 2^10 = 1024).